Informe Lab 4 - Analizador Léxico SQL

Curso: Estructuras de Datos Avanzadas · Práctica de Laboratorio 4

Integrantes:

Diego Canqui Ecos

Diego Mauricio Villanueva Flores

Repositorio: github.com/diego08-hub/Analizador_Lexico_EDA

1. Introducción y objetivo

Se implementó en C++17 un analizador léxico para un subconjunto SQL de sentencias `SELECT` que convierte el texto de la consulta en una secuencia ordenada de tokens con tipo, lexema y posición (línea y columna).

El módulo es independiente del resto del motor: no ejecuta consultas ni comprueba que existan tablas o columnas. Su salida está pensada para el analizador sintáctico de la siguiente etapa.

2. Arquitectura y diseño

El proyecto separa responsabilidades en cinco archivos, cada uno con una función única.

Componente	Archivo	Función
Modelo de tokens	`src/token.h`	`TokenType`, `Token` y conversión del tipo a texto
Tabla de palabras reservadas	`src/keywords.h`	`lookupKeyword`, búsqueda sin distinguir mayúsculas
Reconocedor léxico	`src/lexer.h`, `src/lexer.cpp`	Clase `Lexer` con `tokenize()` y `getErrors()`
Lectura de entrada y salida	`src/main.cpp`	Lee desde archivo o entrada estándar e imprime la tabla de tokens y errores
Pruebas	`tests/test_lexer.cpp`	46 comprobaciones y generación del reporte

El algoritmo recorre la entrada una sola vez, de izquierda a derecha, con lectura de uno o dos caracteres hacia adelante (`peek` y `peekNext`):

1. `skipWhitespace` omite espacios, tabulaciones y saltos de línea.
2. `scanToken` decide por el primer carácter: letra o `_` → `readIdentifier`; dígito → `readNumber`; comilla simple → `readString`; cualquier otro → `readSymbol`.
3. `readIdentifier` consulta la tabla de reservadas con una copia en mayúsculas, sin modificar el lexema original.
4. `readSymbol` resuelve los operadores de dos caracteres mirando el siguiente, por lo que `>=`, `<=`, `<>` y `!=` nunca se dividen.
5. `advance` actualiza línea y columna; la columna vuelve a 1 después de cada `\n`.
6. Al terminar se agrega el token `END_OF_FILE`.

La complejidad es O(n) en tiempo y espacio. `tokenize()` reinicia su estado al empezar, así que puede llamarse varias veces sin duplicar tokens.

3. Modelo de tokens y reglas léxicas

Cada `Token` guarda `type`, `lexeme`, `line`, `column` y `message` (este último solo se llena en los errores). Los nombres de los tokens son propios del equipo y se documentan aquí.

Categoría	Token(s)	Lexema reconocido
Palabras reservadas	`SELECT_KW`, `FROM_KW`, `WHERE_KW`, `AND_KW`, `OR_KW`	SELECT, FROM, WHERE, AND, OR
Identificador	`IDENTIFIER`	`[A-Za-z_][A-Za-z0-9_]*`
Entero	`INTEGER`	`[0-9]+`
Decimal	`DECIMAL`	`[0-9]+\.[0-9]+`
Cadena	`STRING`	`'...'`, el lexema incluye las comillas
Relacionales	`EQUAL`, `NOT_EQUAL`, `LESS`, `GREATER`, `LESS_EQUAL`, `GREATER_EQUAL`	`=`, `<>` y `!=`, `<`, `>`, `<=`, `>=`
Puntuación	`COMMA`, `SEMICOLON`, `STAR`	`, ; *`
Fin de entrada	`END_OF_FILE`	vacío
Error léxico	`ERROR`	texto problemático, con mensaje en `message`

Reglas léxicas adoptadas:

- **Mayúsculas y minúsculas:** las palabras reservadas no distinguen mayúsculas (`select` = `SELECT`). El lexema conserva el texto original.
- **Identificadores:** empiezan con letra o `_`. Una reservada solo se reconoce si coincide completa, por lo que `selected` es un identificador.
- **Espacios en blanco:** espacios, tabulaciones y saltos de línea se ignoran; solo separan tokens y cuentan para la posición.
- **Operadores:** siempre se toma el más largo posible. `<>` y `!=` producen el mismo tipo, `NOT_EQUAL`.
- **Números:** un decimal exige dígitos a ambos lados del punto. `1.` es un error léxico.
- **Cadenas:** entre comillas simples, sin saltos de línea dentro y sin secuencias de escape.
- **Posición:** línea y columna (desde 1) del primer carácter del token. `END_OF_FILE` marca la posición final.

Se agregó `STAR` (`*`) para admitir `SELECT *`, que no estaba en la lista mínima.

4. Manejo de errores léxicos

Un error nunca se clasifica como token válido. En el punto donde ocurre se emite un token `ERROR` con su posición y se guarda el mensaje `Error léxico en línea L, columna C: ...`. El análisis continúa, así que todos los errores se reportan en una sola pasada.

Entrada	Mensaje	Ejemplo y posición
Carácter no admitido (`@`, `#`, `$`, `.` suelto)	carácter no válido	`a @ b` → error en 1:3
`!` sin `=` a continuación	carácter no válido '!'	`!` → error en 1:1
Cadena sin comilla de cierre (hasta fin de línea o de entrada)	cadena sin cerrar	`x = 'abc` → error en 1:5
Decimal sin dígitos después del punto	número decimal mal formado	`1.` → error en 1:1

Limitaciones documentadas: no se admite `tabla.columna` (el `.` solo es válido dentro de un decimal), y `12abc` se separa en `INTEGER(12)` e `IDENTIFIER(abc)`; será el analizador sintáctico quien rechace esa secuencia. Tampoco se admiten comentarios, comillas dobles ni caracteres fuera de ASCII: una letra como `ñ` se reporta como un error por cada byte.

5. Pruebas y resultados

Con `make test` se ejecutan 46 comprobaciones sin dependencias externas: 46 pasan y 0 fallan. El programa también genera `docs/Reporte_Pruebas.md`, con la consulta, los tokens encontrados y los errores de cada prueba.

#	Escenario	Comprobaciones	Casos verificados
1	Sentencia válida	2	`SELECT ... FROM ... WHERE` completo con `AND` y `OR`; `SELECT *`
2	Operadores compuestos	8	`>=`, `<=`, `<>`, `!=` como un solo token; `< =` separados; `>>` como dos tokens
3	Literales	7	entero, decimal, cadena, cadena vacía, cadena con símbolos dentro
4	Reservadas e identificadores	5	mayúsculas y minúsculas, lexema original conservado, `selected` y `fromage` como identificadores, `_id`
5	Espacios en blanco	6	espacios, tabulaciones, `\n`, `\r\n`; posiciones tras salto de línea y tabulación
6	Errores léxicos	9	`@`, `!` solo, `1.`, cadena sin cerrar (no cruza líneas), varios errores seguidos, mensaje con posición
7	Casos límite	9	entrada vacía, solo espacios, sentencias incompletas, token al final, posición de `END_OF_FILE`, `tokenize()` repetido

Ejemplo de ejecución con una consulta válida:

```
None
$ echo "SELECT a FROM t WHERE x >= 1.5 AND y <> 'hi';" | ./lexer
TIPO            LEXEMA              POSICION
SELECT_KW       "SELECT"            1:1
IDENTIFIER      "a"                 1:8
FROM_KW         "FROM"              1:10
IDENTIFIER      "t"                 1:15
WHERE_KW        "WHERE"             1:17
IDENTIFIER      "x"                 1:23
```

```
GREATER_EQUAL   ">="                1:25
DECIMAL         "1.5"               1:28
AND_KW          "AND"               1:32
IDENTIFIER      "y"                 1:36
NOT_EQUAL       "<>"                1:38
STRING          "'hi'"              1:41
SEMICOLON       ";"                 1:45
END_OF_FILE     ""                  2:1
```

Ejemplo con errores, donde el análisis continúa después de cada uno:

None

```
$ printf "SELECT @a, t.b FROM t" | ./lexer
SELECT_KW       "SELECT"            1:1
ERROR           "@"                 1:8     carácter no válido
'@'
IDENTIFIER      "a"                 1:9
COMMA           ","                 1:10
IDENTIFIER      "t"                 1:12
ERROR           "."                 1:13    carácter no válido
'.'
IDENTIFIER      "b"                 1:14
FROM_KW         "FROM"              1:16
IDENTIFIER      "t"                 1:21
END_OF_FILE     ""                  1:22
```

6. Compilación, uso e integración

Se necesita `g++` con soporte para C++17 y `make`.

Shell

```
make              # compila el ejecutable ./lexer
make test         # compila y corre las pruebas, y regenera
docs/Reporte_Pruebas.md
make clean        # borra los binarios

./lexer consulta.sql                          # lee desde un
archivo
```

```
echo "SELECT a FROM t WHERE x >= 1.5;" | ./lexer   # lee desde la entrada estándar
```

La salida es una tabla con tipo, lexema y posición `línea:columna`; si hubo errores léxicos, se listan además al final.

Para integrarlo con el analizador sintáctico basta con incluir `lexer.h`:

```C/C++
Lexer lexer(sql);
std::vector<Token> tokens = lexer.tokenize();   // el último siempre es END_OF_FILE
const auto& errores = lexer.getErrors();        // mensajes legibles
```

El parser puede consumir el vector de forma secuencial y localizar cada error léxico por su token `ERROR`.

7. Conclusiones

El analizador reconoce todas las categorías pedidas, mantiene el orden y la posición de cada token, y detecta los errores léxicos sin clasificarlos como tokens válidos. Las 46 pruebas cubren los siete escenarios del laboratorio: casos válidos, inválidos y límite.

El diseño en mód ulos pequeños (tokens, palabras reservadas, reconocedor, entrada y salida) y una API de dos funciones, `tokenize()` y `getErrors()`, facilitan su integración con el analizador sintáctico.

Como trabajo futuro quedan la notación `tabla.columna`, los comentarios SQL, las comillas dobles y el soporte de UTF-8 en identificadores y cadenas.